

Análisis Exploratorio (EDA)

AUTHOR

Massielle Coti

```
# Cargar librerías R
library(tidyverse)
library(naniar)
library(VIM)
library(mice)
library(ggplot2)
library(dplyr)
library(tidyr)
library(corrplot)
```

```
# Cargar bibliotecas Python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.preprocessing import StandardScaler
import missingno as msno
import warnings
warnings.filterwarnings('ignore')
```

```
# Crear dataset de ejemplo con valores faltantes y atípicos
set.seed(123)
n <- 1000
```

```
datos_r <- data.frame(
  id = 1:n,
  edad = round(rnorm(n, mean = 35, sd = 12)),
  ingreso = round(rnorm(n, mean = 50000, sd = 15000)),
  educacion = sample(1:5, n, replace = TRUE, prob = c(0.1, 0.2, 0.3, 0.25, 0.15)),
  experiencia = round(rnorm(n, mean = 10, sd = 7)),
  satisfaccion = round(runif(n, min = 1, max = 10))
)
```

```
# Introducir valores faltantes (MCAR)
indices_na <- sample(1:n, size=50)
datos_r$edad[indices_na] = NA
```

```
# Introducir valores atípicos
indices_outliers <- sample(1:n, size=20)
datos_r$ingreso[indices_outliers] = datos_r$ingreso[indices_outliers] * 5
```

```
# Introducir valores faltantes condicionales (MAR)
```

```
indices_mar <- sample(which(datos_r$educacion >=4),size = 30)
datos_r$satisfaccion[indices_mar] = NA

head(datos_r)
```

```
# Crear dataset de ejemplo con valores faltantes y atípicos con python
np.random.seed(123)
n = 1000

datos_python = pd.DataFrame({
    'id': range(1,n+1),
    'edad': np.round(np.random.normal(35,12,n)).astype(int),
    'ingreso': np.round(np.random.normal(50000, 15000, n)).astype(int),
    'educacion': np.random.choice([1,2,3,4,5], n, p=[0.1, 0.2, 0.3, 0.25, 0.15]),
    'experiencia': np.round(np.random.normal(10,7,n)).astype(int),
    'satisfaccion': np.round(np.random.uniform(1,10,n))
})

# Introducir valores faltantes (MCAR)
indices_na = np.random.choice(range(n), size=50, replace=False)
datos_python.loc[indices_na, 'edad'] = np.nan

# Introducir valores atípicos
indices_outliers = np.random.choice(range(n), size=20, replace=False)
datos_python.loc[indices_outliers, 'ingreso'] = datos_python.loc[indices_outliers, 'ingreso'] * 5

# Introducir valores faltantes condicionales (MAR)
indices_mar = np.random.choice(datos_python[datos_python['educacion'] >= 4].index, size=30, replace=True)
datos_python.loc[indices_mar, 'satisfaccion'] = np.nan

# Ver resultados
print(datos_python.head())
```

	id	edad	ingreso	educacion	experiencia	satisfaccion
0	1	22.0	38768	3	11	2.0
1	2	47.0	58514	3	14	10.0
2	3	38.0	60772	4	10	8.0
3	4	17.0	35009	4	4	3.0
4	5	28.0	57123	3	14	2.0

Análisis exploratorio inicial

```
#| label: analisis_inicial_r
#| echo: true

# Resumen estadístico
summary(datos_r)
```

```
# Estructura del dataset
str(datos_r)

# Visualización de datos faltantes
gg_miss_var(datos_r) + labs(title = "Proporcion de valores faltantes por variable",
x = "Variable", y="Proporcion de faltantes"
)
```

Equivalente en Python

```
# Resumen estadístico
print(datos_python.describe())

# Información del dataset
print(datos_python.info())

# Visualización de datos faltantes
plt.figure(figsize=(10,6))
msno.bar(datos_python)
plt.title("Proporcion de valores faltantes por variable")
plt.show()
```

	id	edad	ingreso	educacion	experiencia
count	1000.000000	950.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	34.511579	54094.723000	3.180000	10.159000
std	288.819436	11.902633	33976.095671	1.168317	6.874371
min	1.000000	-4.000000	-7021.000000	1.000000	-9.000000
25%	250.750000	27.000000	40551.250000	2.000000	5.000000
50%	500.500000	34.000000	51621.000000	3.000000	10.000000
75%	750.250000	43.000000	60391.500000	4.000000	15.000000
max	1000.000000	78.000000	441690.000000	5.000000	31.000000

 satisfaccion

count	970.000000
mean	5.583505
std	2.570319
min	1.000000
25%	3.000000
50%	6.000000
75%	8.000000
max	10.000000

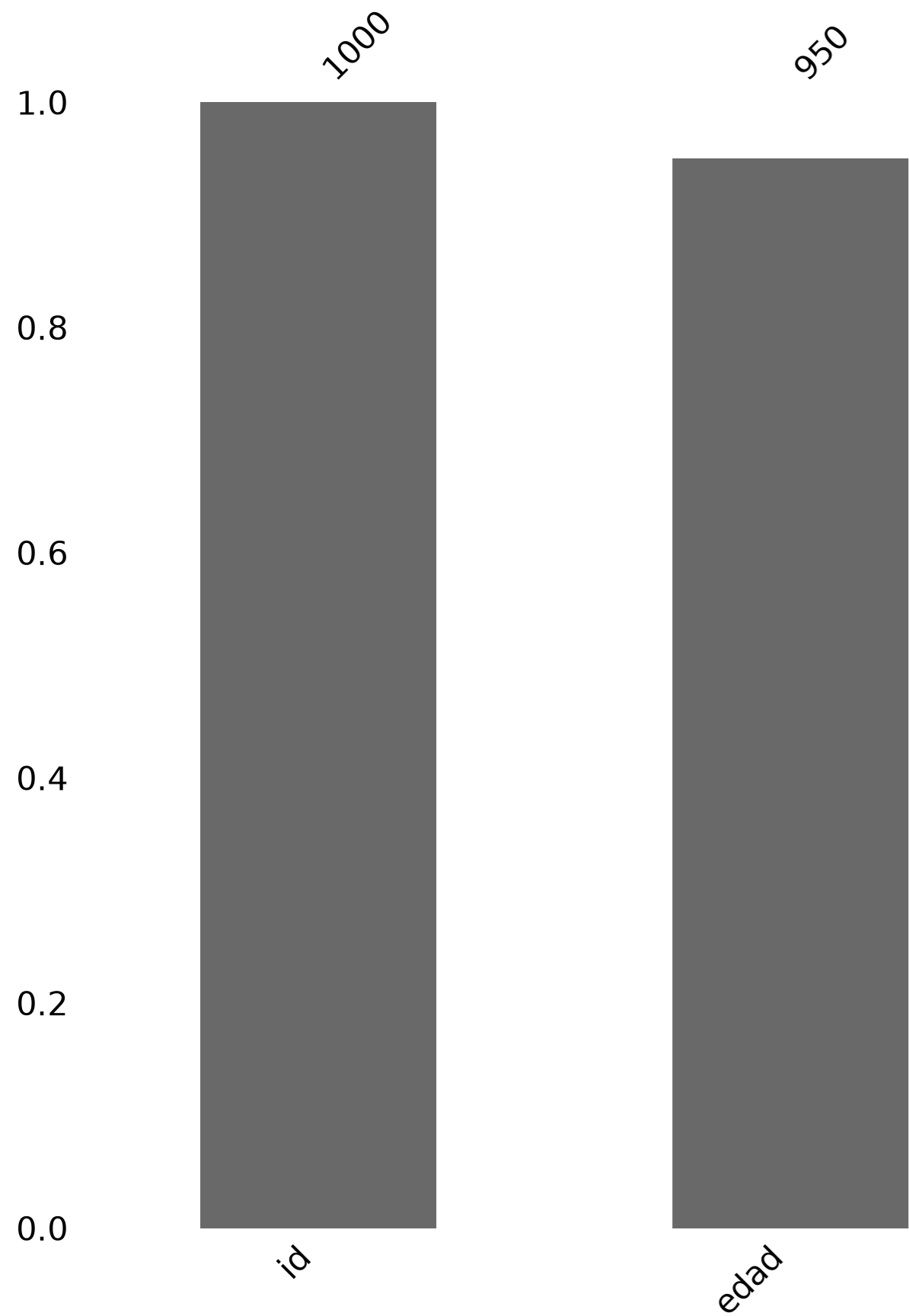
<class 'pandas.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	id	1000 non-null	int64
1	edad	950 non-null	float64
2	ingreso	1000 non-null	int64
3	educacion	1000 non-null	int64

```
4  experiencia  1000 non-null  int64
5  satisfaccion 970 non-null   float64
dtypes: float64(2), int64(4)
memory usage: 47.0 KB
None
```



Imputacion de valores faltantes

Metodos de imputacion

Tipos de imputacion 1. Imputacion simple: - Media / Mediana / Moda - Valor constante - Ultimo valor observado (LOCF) 2. Imputacion avanzada: - Regresion - K-Nearest (KNN) - Imputacion multiple por ecuaciones encadenadas (MICE) - Imputacion por arboles de decision

Imputacion en R

```
#| label: imputacion_r
#| echo: true

# Metodo 1: Imputacion por mediana
datos_r_mediana = datos_python
datos_r_mediana$edad[in_na(datos_r_mediana$edad)] <- median(datos_r_mediana$edad, na.rm = TRUE)

datos_r_mediana$satisfaccion[is_na(datos_r_mediana$satisfaccion)] <-
median(datos_r_mediana$satisfaccion, na.rm = TRUE)

# Metodo 2: Imputacion por Regresion Lineal
modelo_edad <- lm(edad ~ ingreso + educacion + experiencia, datos_r)
datos_r_lm <- datos_r_
inice_na_edad <- which(is_na(datos_r_lm$edad))
if(length(indices_na_edad)>0) {
  datos_r_lm$edad[indices_na_edad] <- predict(modelo_edad, newdata =
datos_r_lm[indices_na_edad,])
}

# Metodo 3: Imputacion multiple MICE
imputaciones <- mice(datos_r, m=5, method = 'pmm', maxit = 50, seed = 123)
datos_r_mice <- complete(imputaciones,1)

# Comparacion de los metodos de imputacion
comparacion <- data.frame(Variable = c("Edad", "Satisfaccion"),
  Original_NA = c(sum(is_na(datos_r$edad)),
    sum(is_na(datos_r$satisfaccion))),
  Mediana_NA = c(sum(is_na(datos_r_mediana$edad)),
    sum(is_na(datos_r_mediana$satisfaccion))),
  LM_NA = c(sum(is_na(datos_r_lm$edad)),
    sum(is_na(datos_r_lm$satisfaccion))),
  MICE_NA = c(sum(is_na(datos_r_mice$edad)),
    sum(is_na(datos_r_mice$satisfaccion)))
)

print(comparacion)
```

```
print(datos_r[indices_na_edad, ])
datos_r_mediana[indices_na_edad, ]
datos_r_lm[indices_na_edad, ]
datos_r_mice[indices_na_edad, ]
```

Imputación en Python

```
from sklearn.linear_model import LinearRegression

# Método 1: Imputación por mediana
imputer_median = SimpleImputer(strategy='median')
datos_py_median = pd.DataFrame(imputer_median.fit_transform(datos_python),
                               columns=datos_python.columns)

# Método 2: Imputación por KNN
imputer_knn = KNNImputer(n_neighbors=5)
datos_py_knn = pd.DataFrame(imputer_knn.fit_transform(datos_python),
                             columns=datos_python.columns)

# Método 3: Imputación por regresión (solo para edad)
datos_py_lm = datos_python.copy()
edad_na_mask = datos_py_lm['edad'].isna()
if edad_na_mask.any():
    train_data = datos_py_lm[~edad_na_mask]
    test_data = datos_py_lm[edad_na_mask]

    X_train = train_data[['ingreso', 'educacion', 'experiencia']].values
    y_train = train_data['edad'].values
    X_test = test_data[['ingreso', 'educacion', 'experiencia']].values

    model = LinearRegression()
    model.fit(X_train, y_train)
    datos_py_lm.loc[edad_na_mask, 'edad'] = model.predict(X_test)

# Comparación de métodos
comparacion_py = pd.DataFrame({
    'Variable': ['Edad', 'Satisfacción'],
    'Original_NA': [datos_python['edad'].isna().sum(), datos_python['satisfaccion'].isna().sum()],
    'Mediana_NA': [datos_py_median['edad'].isna().sum(), datos_py_median['satisfaccion'].isna().sum()],
    'KNN_NA': [datos_py_knn['edad'].isna().sum(), datos_py_knn['satisfaccion'].isna().sum()],
    'LM_NA': [datos_py_lm['edad'].isna().sum(), datos_py_lm['satisfaccion'].isna().sum()]
})
print(comparacion_py)
```

	Variable	Original_NA	Mediana_NA	KNN_NA	LM_NA
0	Edad	50	0	0	0
1	Satisfacción	30	0	0	30

Detección y Tratamiento en Python

```
import matplotlib.pyplot as plt

# Función para detectar outliers usando IQR
def detectar_outliers_iqr(df, columna):
    Q1 = df[columna].quantile(0.25)
    Q3 = df[columna].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return (df[columna] < lower_bound) | (df[columna] > upper_bound)

# Detectar outliers en ingreso
outliers_ingreso = detectar_outliers_iqr(datos_python, 'ingreso')
print(f"Número de outliers en ingreso: {outliers_ingreso.sum()}")

# Boxplot para visualizar datos usando matplotlib
plt.figure(figsize=(10, 6))
plt.boxplot(datos_python['ingreso'])
plt.title('Boxplot de ingreso - Detección de outliers')
plt.ylabel('Ingreso')
plt.show()

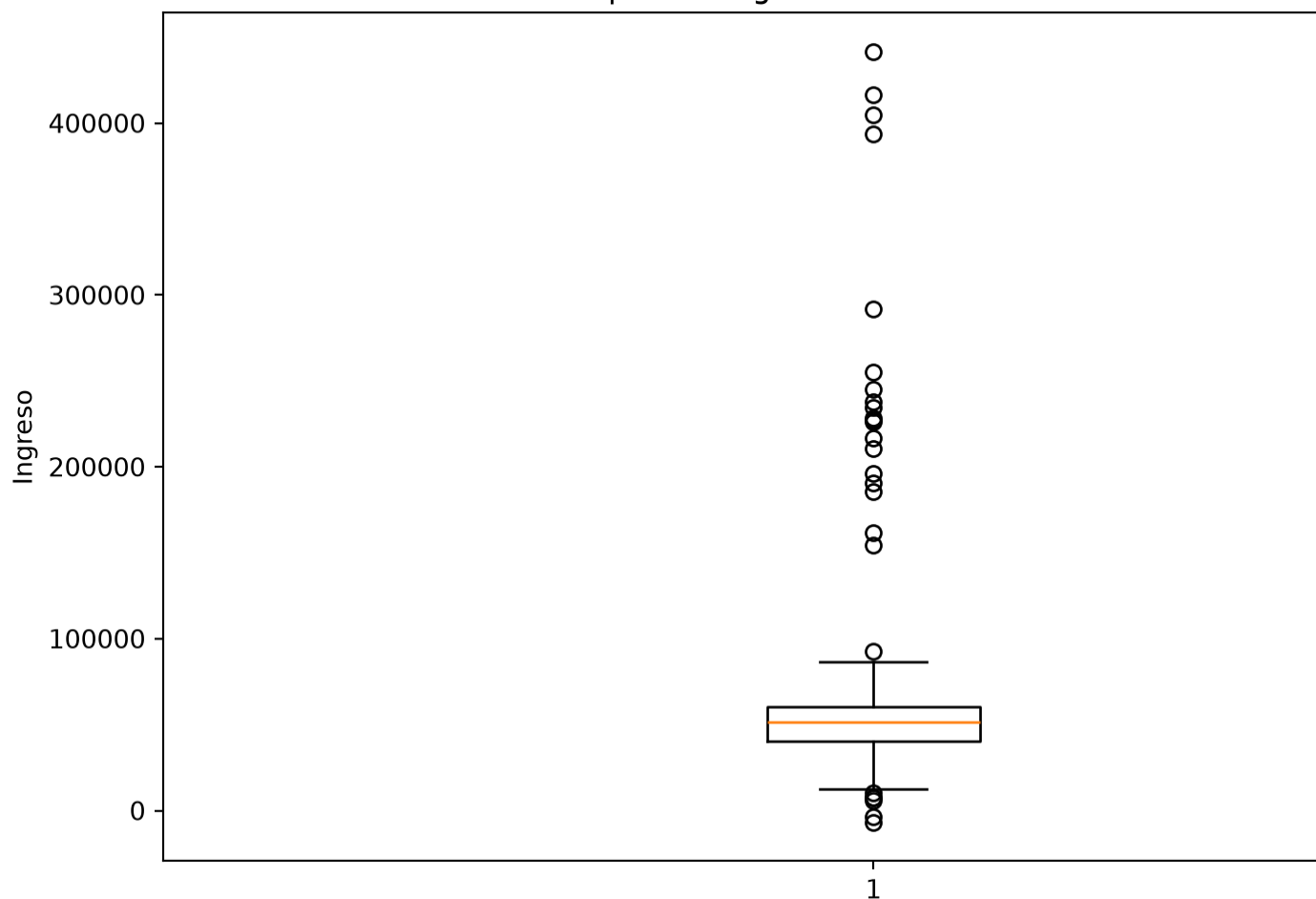
# Tratamiento de outliers: Winsorización
def winsorizar(serie, lower_percentile=0.05, upper_percentile=0.95):
    lower = serie.quantile(lower_percentile)
    upper = serie.quantile(upper_percentile)
    return serie.clip(lower=lower, upper=upper)

datos_py_winsor = datos_python.copy()
datos_py_winsor['ingreso'] = winsorizar(datos_py_winsor['ingreso'])

# Comparación antes/después de winsorización
print("Estadísticos originales:")
print(datos_python['ingreso'].describe())
print("\nEstadísticos después de winsorización:")
print(datos_py_winsor['ingreso'].describe())
```

Número de outliers en ingreso: 26

Boxplot de ingreso - Detección de outliers



Estadísticos originales:

```
count    1000.000000
mean      54094.723000
std       33976.095671
min       -7021.000000
25%       40551.250000
50%       51621.000000
75%       60391.500000
max       441690.000000
```

Name: ingreso, dtype: float64

Estadísticos después de winsorización:

```
count    1000.000000
mean      50753.654500
std       13506.660229
min       25787.100000
25%       40551.250000
50%       51621.000000
75%       60391.500000
max       74838.450000
```

Name: ingreso, dtype: float64

```
# Función para detectar outliers usando IQR
def detectar_outliers_iqr(df, columna):
    Q1 = df[columna].quantile(0.25)
    Q3 = df[columna].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return (df[columna] < lower_bound) | (df[columna] > upper_bound)

# Detectar outliers en ingreso
outliers_ingreso = detectar_outliers_iqr(datos_python, 'ingreso')
print(f"Número de outliers en ingreso: {outliers_ingreso.sum()}")

# Boxplot para visualizar outliers
plt.figure(figsize=(10, 6))
plt.boxplot(datos_python['ingreso'])
plt.title('Boxplot de Ingreso - Detección de Outliers')
plt.ylabel('Ingreso')
plt.show()

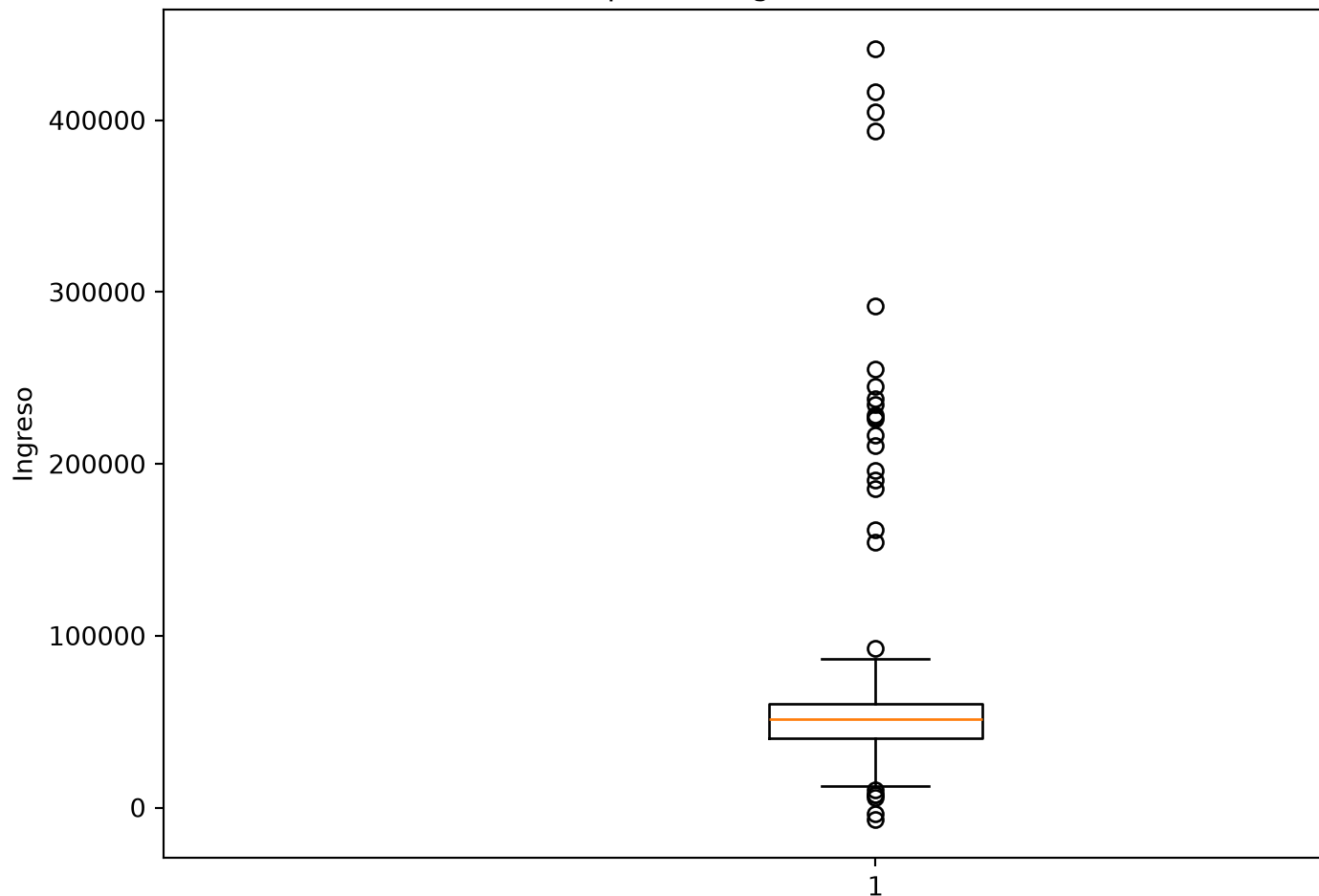
# Tratamiento de outliers: Winsorización
def winsorizar(serie, lower_percentile=0.05, upper_percentile=0.95):
    lower = serie.quantile(lower_percentile)
    upper = serie.quantile(upper_percentile)
    return serie.clip(lower=lower, upper=upper)

datos_py_winsor = datos_python.copy()
datos_py_winsor['ingreso'] = winsorizar(datos_py_winsor['ingreso'])

# Comparación antes/después de winsorización
print("Estadísticos originales:")
print(datos_python['ingreso'].describe())
print("\nEstadísticos después de winsorización:")
print(datos_py_winsor['ingreso'].describe())
```

Número de outliers en ingreso: 26

Boxplot de Ingreso - Detección de Outliers



Estadísticos originales:

```
count    1000.000000
mean      54094.723000
std       33976.095671
min       -7021.000000
25%       40551.250000
50%       51621.000000
75%       60391.500000
max       441690.000000
```

Name: ingreso, dtype: float64

Estadísticos después de winsorización:

```
count    1000.000000
mean      50753.654500
std       13506.660229
min       25787.100000
25%       40551.250000
50%       51621.000000
75%       60391.500000
max       74838.450000
```

Name: ingreso, dtype: float64

Analisis de resultados y visualizacion

Comparacion de metodos de imputacion

```
# Función para calcular estadísticos comparativos
def comparar_imputaciones(original, imputado, variable):
    return pd.DataFrame({
        'Métrica': ['Media', 'Mediana', 'Desv_Estandar'],
        'Original': [
            original[variable].mean(),
            original[variable].median(),
            original[variable].std()
        ],
        'Imputado': [
            imputado[variable].mean(),
            imputado[variable].median(),
            imputado[variable].std()
        ]
    })

# Comparar métodos para edad
print("Comparación de métodos de imputación para Edad:")
comparacion_py_median = comparar_imputaciones(datos_python, datos_py_median, 'edad')
print("\nImputación por Mediana:")
print(comparacion_py_median)

comparacion_py_knn = comparar_imputaciones(datos_python, datos_py_knn, 'edad')
print("\nImputación por KNN:")
print(comparacion_py_knn)

comparacion_py_lm = comparar_imputaciones(datos_python, datos_py_lm, 'edad')
print("\nImputación por Regresión:")
print(comparacion_py_lm)

# Visualización de distribución
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].hist(datos_python['edad'].dropna(), bins=30, color='lightblue')
axes[0, 0].set_title('Edad Original')
axes[0, 1].hist(datos_py_median['edad'], bins=30, color='lightgreen')
axes[0, 1].set_title('Edad - Mediana')
axes[1, 0].hist(datos_py_knn['edad'], bins=30, color='lightyellow')
axes[1, 0].set_title('Edad - KNN')
axes[1, 1].hist(datos_py_lm['edad'], bins=30, color='lightpink')
axes[1, 1].set_title('Edad - Regresión')
plt.tight_layout()
plt.show()
```

Comparación de métodos de imputación para Edad:

Imputación por Mediana:

	Métrica	Original	Imputado
0	Media	34.511579	34.486000
1	Mediana	34.000000	34.000000
2	Desv_Estandar	11.902633	11.601483

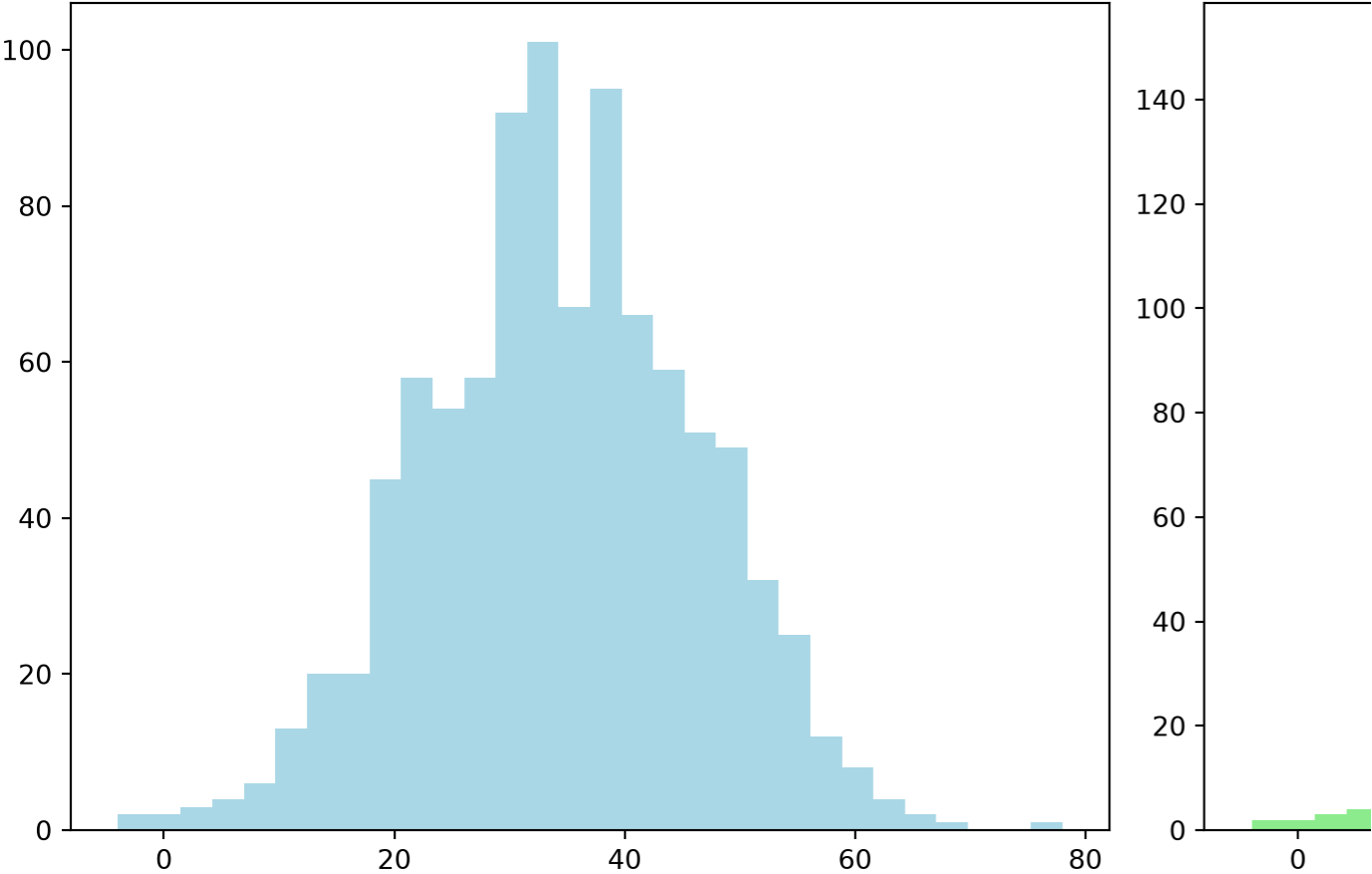
Imputación por KNN:

	Métrica	Original	Imputado
0	Media	34.511579	34.508800
1	Mediana	34.000000	34.000000
2	Desv_Estandar	11.902633	11.673129

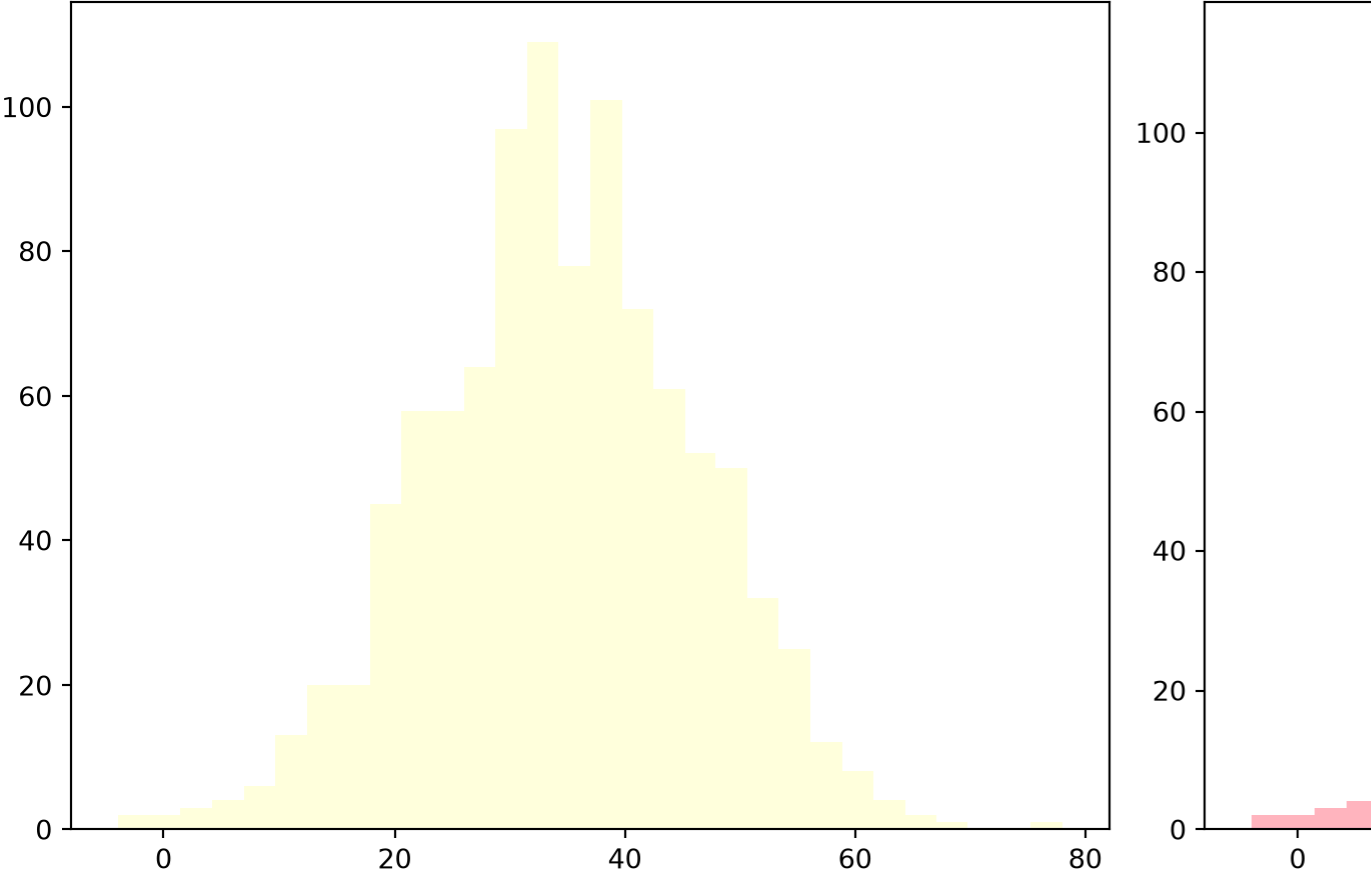
Imputación por Regresión:

	Métrica	Original	Imputado
0	Media	34.511579	34.514808
1	Mediana	34.000000	34.465070
2	Desv_Estandar	11.902633	11.601306

Edad Original



Edad - KNN



Analisis de Correlacion y Relaciones

Correlacion y relaciones en R

```
#| label: analisis-correlacion-r
#| echo: True

# Matriz de correlacion despues de imputacion
correlacion <- cor(datos_r_mice[, c("edad", "ingreso", "educacion", "experiencia",
"satisfaccion")])

# Visualizacion de la matriz de correlacion
corrplot(correlacion, method = "color", type = "upper",
addCoef.col = "black", tl.col = "black",
title = "Matriz de Correlacion", mar = c(0,0,2,0))

# Grafico de dispersion
pairs(datos_r_mice[, c("edad", "ingreso", "educacion", "experiencia")],
main = "Graficos de Dispersion entre Variables")
```

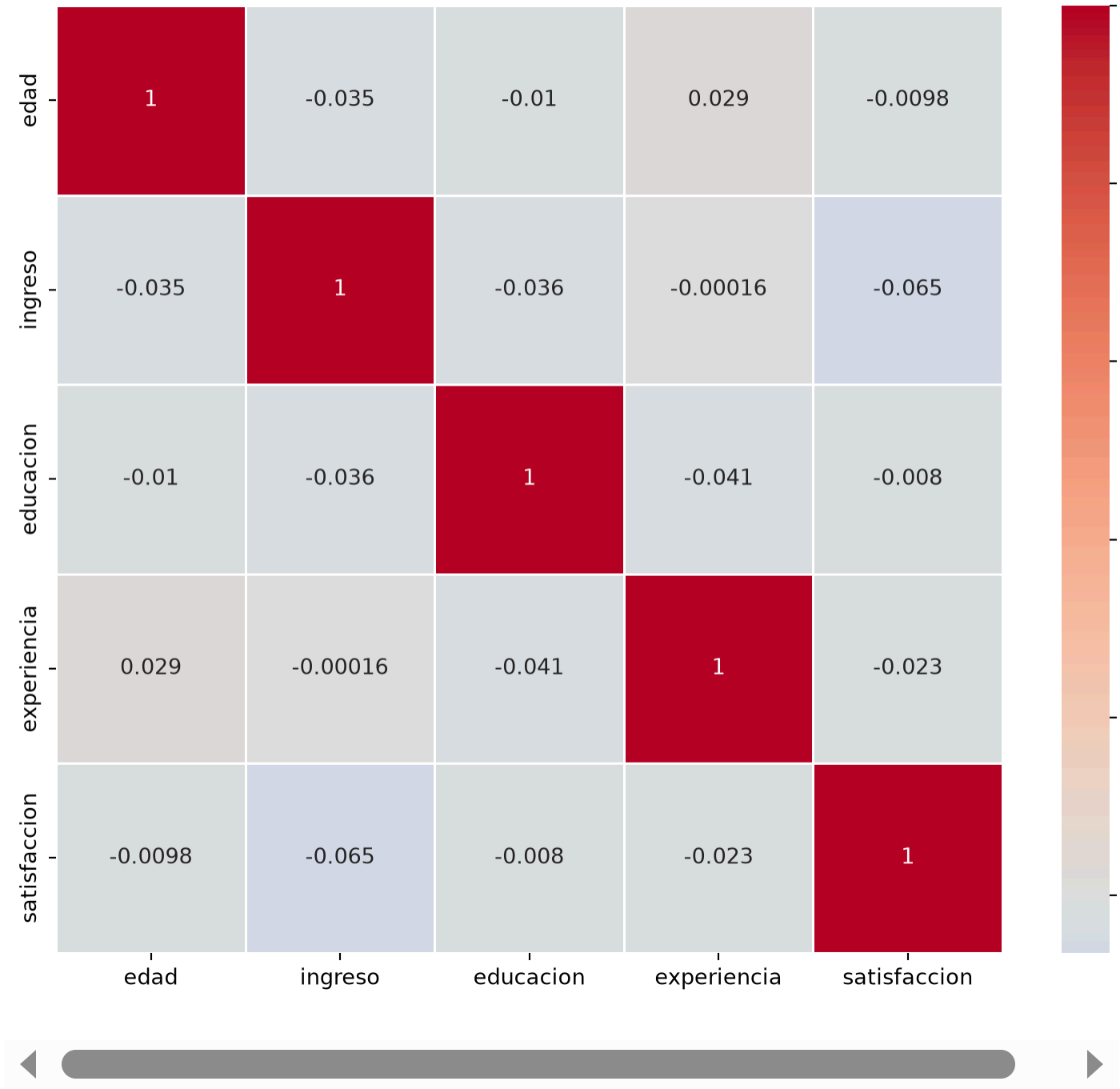
Correlacion y relaciones en Python

```
# Matriz de correlación después de imputación
correlacion = datos_py_lm[['edad', 'ingreso', 'educacion', 'experiencia', 'satisfaccion']].corr()

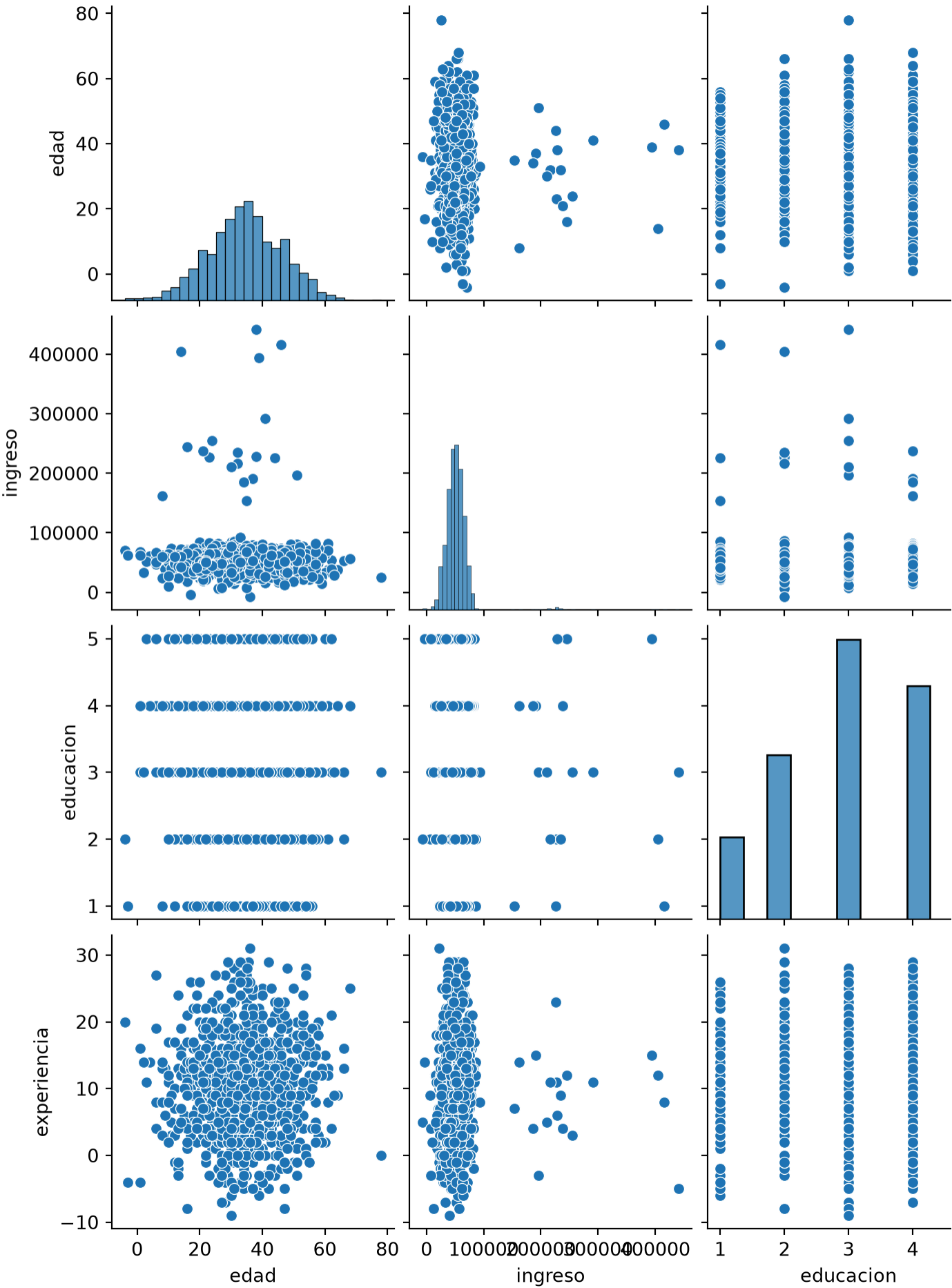
# Visualización de la matriz de correlación
plt.figure(figsize=(10, 8))
sns.heatmap(correlacion, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5)
plt.title('Matriz de Correlación')
plt.show()

# Pairplot
sns.pairplot(datos_py_lm[['edad', 'ingreso', 'educacion', 'experiencia']])
plt.suptitle('Gráficos de Dispersión entre Variables', y=1.02)
plt.show()
```

Matriz de Correlación



Gráficos de Dispersión entre Variables





Conclusiones y recomendaciones

Resumen de hallazgos

Resumen R

```
#| label: conclusiones-r
#| echo: true

# Resumen final
cat("RESUMEN DEL ANALISIS EXPLORATORIO\n\n")
cat("1. Datos faltantes:\n")
cat("  - Edad: MCAR (50 valores faltantes)\n")
cat("  - Satisfaccion: MAR (30 valores faltantes en educacion >=4)\n\n")

cat("2. Metodos de imputacion evaluados:\n")
cat("  - Mediana: Simple pero puede sesgar la distribucion\n")
cat("  - Regresion: Considera relaciones variables\n")
cat("  - MICE/KNN: Captura patrones complejos en los datos\n\n")

cat("3. Outliers detectados:\n")
cat("  - Ingreso: 20 valores extremos identificados\n")
cat("  - Tratamiento aplicado: Winsorizacion al 5% y 95%\n\n")

cat("4. Recomendaciones:\n")
cat("  - Para datos MCAR: Cualquier metodo es valido\n")
cat("  - Para datos MAR: Usar metodos avanzados como MICE\n")
cat("  - Para outliers: Winsorizacion o transformacion logaritmica\n\n")
```

Guardado y carga de datos

```
#| label: guardar-cargar-datos
#| echo: true

# Guardar los datos generados
write.csv(datos_r, "datos_ejemplo.csv", row.names = FALSE) # df. to csv('nombre.csv',
index=False)

# Funcion para cargar datos (si ya existen)
cargar_datos <- function(ruta = "datos_ejemplo.csv"){
  if (file.exists(ruta)){
    datos <- read.csv(ruta)
    cat("Datos cargados desde:", ruta, "\n")
  } else {
    cat("No se encontro el archivo. Generando datos nuevos...\n")
    #Codigo para generar datos (el mismo quea antes)
```

```
set.seed(123)
n <- 1000

datos <- data.frame(
  id = 1:n,
  edad = round(rnorm(n, mean = 35, sd = 12)),
  ingreso = round(rnorm(n, mean = 50000, sd = 15000)),
  educacion = sample(1:5, n, replace = TRUE, prob = c(0.1, 0.2, 0.3, 0.25, 0.15)),
  experiencia = round(rnorm(n, mean = 10, sd = 7)),
  satisfaccion = round(runif(n, min = 1, max = 10))
)

#Introducir valores faltantes y outliers...
write.csv(datos, ruta, row.names = FALSE)
}
return(datos)

}

datos_r <- cargar_datos()
```